

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Experiments

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

FACULTY : Dr. D.SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I GUZHALINITHA.K - 192425232 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

QUESTIONS:10

Total Marks:50

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

6. Perceptron and Multilayer Perceptron (MLP)

- a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.
- b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

7. Forward Propagation and Backpropagation Algorithm

- a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.
- b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

- a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.
- b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Experiment 1.

Experiment 1: Learning Algorithms – Linear Regression using Gradient Descent

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Sample Dataset
```

```
X = np.array([1, 2, 3, 4, 5], dtype=float)
```

```
Y = np.array([2, 4, 5, 4, 5], dtype=float)
```

```
# Initialize parameters
```

```
m = 0
```

```
c = 0
```

```
learning_rate = 0.01
```

```
epochs = 1000
```

```
n = len(X)
```

```
loss_history = []
```

```
# Gradient Descent
```

```
for i in range(epochs):
```

```
    y_pred = m * X + c
```

```
    dm = (-2 / n) * np.sum(X * (Y - y_pred))
```

```
    dc = (-2 / n) * np.sum(Y - y_pred)
```

```
    m = m - learning_rate * dm
```

```
    c = c - learning_rate * dc
```

```
    loss = np.mean((Y - y_pred) ** 2)
```

```
    loss_history.append(loss)
```

```
print("Slope (m):", m)
```

```
print("Intercept (c):", c)
```

```
# Plot Regression Line
```

```
plt.scatter(X, Y, color='blue', label='Data Points')
```

```
plt.plot(X, m * X + c, color='red', label='Regression Line')
```

```
plt.xlabel("X")
```

```
plt.ylabel("Y")
```

```
plt.title("Linear Regression using Gradient Descent")
```

```
plt.legend()
```

```
plt.show()
```

```
# Plot Learning Curve
```

```
plt.plot(loss_history)
```

```
plt.xlabel("Iterations")
```

```
plt.ylabel("Loss")
```

```
plt.title("Learning Curve")
```

```
plt.show()
```

Experiment 2: Maximum Likelihood Estimation (MLE)

```
import numpy as np

np.random.seed(0)

true_mean = 10
true_variance = 4

data = np.random.normal(true_mean, np.sqrt(true_variance), 1000)

estimated_mean = np.mean(data)
estimated_variance = np.mean((data - estimated_mean) ** 2)

print("Actual Mean:", true_mean)
print("Estimated Mean:", estimated_mean)

print("Actual Variance:", true_variance)
print("Estimated Variance:", estimated_variance)
```

Experiment 3: Building Machine Learning Algorithms

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Experiment 4: Neural Network

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

X, y = make_moons(n_samples=500, noise=0.2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = MLPClassifier(hidden_layer_sizes=(10,),
                      max_iter=1000,
                      random_state=42)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
```

Experiment 5: Artificial Neurons

```
import numpy as np

inputs = np.array([1, 2, 3])

weights = np.array([0.5, -0.6, 0.8])
```

```
bias = 0.2
```

```
net = np.dot(inputs, weights) + bias
```

```
sigmoid = 1 / (1 + np.exp(-net))
```

```
relu = max(0, net)
```

```
print("Net Input:", net)
```

```
print("Sigmoid Output:", sigmoid)
```

```
print("ReLU Output:", relu)
```

Experiment 6(a): Perceptron

```
from sklearn.linear_model import Perceptron
```

```
from sklearn.datasets import make_classification, make_circles
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
```

```
X1, y1 = make_classification(
```

```
    n_samples=500,
```

```
    n_features=2,
```

```
    n_redundant=0,
```

```
    n_clusters_per_class=1,
```

```
    random_state=42)
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X1, y1, test_size=0.2, random_state=42)
```

```
model = Perceptron()
```

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

```
print("Linear Dataset Accuracy:", accuracy_score(y_test, pred))
```

```
X2, y2 = make_circles(n_samples=500, noise=0.1)

X_train, X_test, y_train, y_test = train_test_split(
    X2, y2, test_size=0.2)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Non-linear Dataset Accuracy:", accuracy_score(y_test, pred))
```

Experiment 6(b): Multilayer Perceptron

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

iris = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)

per = Perceptron()

per.fit(X_train, y_train)

pred1 = per.predict(X_test)

mlp = MLPClassifier(hidden_layer_sizes=(20,),
                    max_iter=1000,
                    random_state=42)

mlp.fit(X_train, y_train)

pred2 = mlp.predict(X_test)
```

```
print("Perceptron Accuracy:", accuracy_score(y_test, pred1))
```

```
print("MLP Accuracy:", accuracy_score(y_test, pred2))
```

Experiment 7(a): Forward Propagation

```
import numpy as np
```

```
inputs = np.array([1, 2])
```

```
weights = np.array([[0.5, 0.2],  
                    [0.3, 0.8]])
```

```
bias = np.array([0.1, 0.1])
```

```
hidden = np.dot(inputs, weights) + bias
```

```
output = 1 / (1 + np.exp(-hidden))
```

```
print("Hidden Layer Output:")
```

```
print(output)
```

Experiment 7(b): Backpropagation

```
import numpy as np
```

```
x = np.array([[1]])
```

```
y = np.array([[0]])
```

```
w = np.array([[0.5]])
```

```
b = np.array([[0.1]])
```

```
lr = 0.1
```

```
z = np.dot(x, w) + b
```

```
a = 1 / (1 + np.exp(-z))
```

```
error = a - y
```

```
dw = np.dot(x.T, error)
```

```
db = error
```

```
w = w - lr * dw
```

```
b = b - lr * db
```

```
print("Updated Weight:", w)
```

```
print("Updated Bias:", b)
```

Experiment 8(a): Gradient Descent

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
x = 10
```

```
lr = 0.1
```

```
history = []
```

```
for i in range(50):
```

```
    grad = 2 * x
```

```
    x = x - lr * grad
```

```
    history.append(x)
```

```
plt.plot(history)
```

```
plt.xlabel("Iterations")
```

```
plt.ylabel("x")
```

```
plt.title("Gradient Descent")
```

```
plt.show()
```

Experiment 8(b): Stochastic Gradient Descent

```
from sklearn.datasets import make_regression
```

```
from sklearn.linear_model import SGDRegressor, LinearRegression
```

```
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

X, y = make_regression(
    n_samples=500,
    n_features=1,
    noise=10,
    random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

sgd = SGDRegressor(max_iter=1000)

sgd.fit(X_train, y_train)

pred1 = sgd.predict(X_test)

batch = LinearRegression()

batch.fit(X_train, y_train)

pred2 = batch.predict(X_test)

print("SGD MSE:", mean_squared_error(y_test, pred1))

print("Batch GD MSE:", mean_squared_error(y_test, pred2))
```

Experiment 9: Mini-Batch Gradient Descent

```
import numpy as np

X = np.random.rand(100, 1)

y = 3 * X + 2 + np.random.randn(100, 1) * 0.1

w = np.random.randn(1, 1)

b = 0
```

```
lr = 0.1
```

```
batch_size = 10
```

```
for epoch in range(100):
```

```
    for i in range(0, len(X), batch_size):
```

```
        X_batch = X[i:i + batch_size]
```

```
        y_batch = y[i:i + batch_size]
```

```
        pred = X_batch.dot(w) + b
```

```
        dw = -2 * X_batch.T.dot(y_batch - pred) / len(X_batch)
```

```
        db = -2 * np.mean(y_batch - pred)
```

```
        w -= lr * dw
```

```
        b -= lr * db
```

```
print("Weight:", w)
```

```
print("Bias:", b)
```

Experiment 10: Curse of Dimensionality

```
import numpy as np
```

```
from sklearn.datasets import make_classification
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
dimensions = [2, 5, 10, 20, 50]
```

```
for d in dimensions:
```

```
    X, y = make_classification(
```

```
        n_samples=1000,
```

```
        n_features=d,
```

```
n_informative=2,  
n_redundant=0,  
random_state=42)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42)
```

```
model = KNeighborsClassifier()
```

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

```
distance = np.mean(  
    np.linalg.norm(X_train[:100] - X_train[100:200], axis=1))
```

```
print("Dimensions:", d)
```

```
print("Average Distance:", distance)
```

```
print("Accuracy:", accuracy_score(y_test, pred))
```

```
print("-----")
```